

Universitat Politècnica de Catalunya

FACULTAT D'INFORMÀTICA DE BARCELONA

SISTEMA MULTI-ROBOT D'ATRAPA LA
BANDERA

Robòtica Mòbil / Robòtica Avançada

Autors:

Paula Esteve, Sergi Flores, Claudia Gallego, Pau
Gàlvez i Chen Yufeng

24 de juny de 2026

Índex

1	Abast del document	3
2	Context i objectius de la simulació	4
2.1	Objectiu general	4
3	Estructura del projecte de simulació	4
3.1	Inventari complet del repositori	5
4	Execució de la simulació	11
4.1	Requisits	11
4.2	Comanda principal	11
5	Arquitectura de la simulació	12
5.1	Visió global	12
5.2	Diagrama de nodes	12
5.3	Diagrama de flux funcional	14
5.4	Diagrama de la màquina d'estats	16
6	Nodes de la simulació	18
6.1	Node de simulació i spawn dels robots	18
6.2	Filtre de làser	18
6.3	GMapping i mapes individuals	19
6.4	Fusió de mapes	19
6.5	Navegació amb <code>move_base</code>	19
6.6	Coordinador de robots	19
6.7	Detector de bandera	20
6.8	Coordinador CTF	20
6.9	Detall intern del coordinador CTF	21
6.10	Mòduls Python auxiliars	21
7	Algoritmes i decisions de disseny	23
7.1	Exploració per fronteres	23
7.2	Repartiment entre robots	23
7.3	Detecció de color i estimació geomètrica	24
7.4	Condició de captura	24
7.5	Persecució i intercepció	24
8	Paràmetres de simulació	25
8.1	Paràmetres del launch principal	25
8.2	Paràmetres de GMapping	25
8.3	Paràmetres de <code>move_base</code>	26

8.4	Costmaps	27
8.5	Planificador global	27
8.6	DWA local planner	27
8.7	Visió	28
8.8	Paràmetres del joc CTF	29
9	Funcionament complet de la partida	30
9.1	Inici	30
9.2	Exploració	30
9.3	Aproximació i captura	30
9.4	Persecució	30
10	Execució en robot real	31
10.1	Diferències principals respecte a la simulació	31
10.2	Launch principal del robot real	32
10.3	Detecció de bandera amb ArUco/AprilTag	32
10.4	Estimació de posició de la bandera	33
10.5	Temes de càmera en el robot real	33
10.6	Adaptacions de TF	34
10.7	Navegació i coordinació entre robots	35
10.8	Ordre d'execució recomanat	35
11	Conclusió	36

1 Abast del document

Aquest document descriu el projecte `ctf_navigation` en el seu conjunt, incloent tant la part de simulació en Gazebo com la seva execució sobre robots TurtleBot3 Waffle Pi reals. El projecte té dues branques de treball diferenciades: una orientada a un entorn simulat i una altra adaptada a l'entorn físic. Tot i que comparteixen la mateixa idea general i una arquitectura semblant, cada branca presenta particularitats pròpies que cal documentar per separat.

La part de simulació permet desenvolupar i validar el comportament global del sistema en un entorn controlat. En Gazebo es poden crear els robots, definir el món, situar la bandera, repetir proves i observar amb facilitat els topics, els frames TF, els mapes i els objectius de navegació. Aquesta fase és especialment útil per comprovar la integració entre SLAM, fusió de mapes, navegació, detecció visual, coordinació entre robots i lògica del joc.

La part d'execució real trasllada aquesta arquitectura als TurtleBot3 Waffle Pi físics. En aquest cas apareixen factors que en simulació estan simplificats o controlats, com la qualitat de la xarxa, la sincronització entre màquines, la calibració dels sensors, l'estat de la bateria, la il·luminació, el soroll de les mesures i les limitacions físiques de l'espai. Per això, el document també recull els aspectes necessaris per entendre com s'executa el sistema en robots reals i quines diferències apareixen respecte a Gazebo.

El sistema implementa un joc d'atrapa la bandera amb dos robots TurtleBot3 Waffle Pi. Els robots comencen en un entorn que no coneixen completament, construeixen mapa amb SLAM, comparteixen la informació mitjançant fusió de mapes, busquen una bandera amb la càmera i, quan un d'ells la captura, s'activa una fase de persecució. El robot captador passa a ser el portador i intenta tornar a la seva base; l'altre robot passa a ser el perseguidor i intenta interceptar-lo.

La idea més important del projecte no és un únic algoritme aïllat, sinó la integració de diferents components de robòtica mòbil. El sistema combina percepció, mapatge, planificació, visió, coordinació multi-robot i una màquina d'estats de joc. Aquesta integració es prova primer en simulació i després s'adapta a l'execució real, cosa que permet comparar el comportament idealitzat de Gazebo amb les dificultats pròpies d'un entorn físic.

2 Context i objectius de la simulació

2.1 Objectiu general

L'objectiu general del projecte és desenvolupar i documentar un sistema multi-robot capaç de jugar a atrapa la bandera amb dos TurtleBot3 Waffle Pi, tant en simulació com en execució real. El sistema ha de permetre que els robots explorin un entorn desconegut, construeixin i comparteixin mapa, localitzin una bandera mitjançant visió artificial i canviïn de comportament segons l'estat de la partida.

En la fase inicial, els robots han de cooperar. Aquesta cooperació consisteix a compartir la informació del mapa i repartir-se l'exploració per evitar que tots dos intentin descobrir exactament les mateixes zones. En canvi, quan un robot captura la bandera, el comportament passa a ser competitiu: el robot capturador es converteix en portador i intenta tornar a la seva base, mentre que l'altre robot passa a ser perseguidor i intenta interceptar-lo.

La simulació en Gazebo serveix per validar aquesta arquitectura en un entorn controlat. Permet reiniciar proves, moure la bandera, canviar el món, observar els temes de ROS i ajustar paràmetres de navegació, SLAM, detecció visual i coordinació sense els riscos del robot físic. Aquesta fase és útil per comprovar que la lògica general funciona abans de traslladar-la als robots reals.

L'execució real té com a objectiu comprovar fins a quin punt el comportament validat en simulació es manté en un entorn físic. En aquest cas cal tenir en compte factors com la connexió entre màquines, la sincronització de TF, el soroll dels sensors, la il·luminació, la bateria, les limitacions de moviment i la seguretat. Per tant, el projecte no només busca que el sistema funcioni en Gazebo, sinó també analitzar les adaptacions necessàries perquè pugui executar-se sobre robots TurtleBot3 Waffle Pi reals.

3 Estructura del projecte de simulació

El paquet ROS es diu `ctf_navigation`. La carpeta conté fitxers de compilació, llançadors, configuracions, scripts Python, codi C++ i recursos de simulació. En aquesta memòria no es descriuen tots els fitxers línia per línia, sinó que s'expliquen segons la seva funció dins del sistema.

Taula 1: Organització funcional de la part de simulació

Bloc	Fitxers principals	Paper dins del sistema
Paquet ROS	<code>package.xml</code> , <code>CMakeLists.txt</code>	Declaren dependències, executables i instal·lació del paquet.

Bloc	Fitxers principals	Paper dins del sistema
Llançament principal	<code>launch/slam_multi_robot_ctf.launch</code>	Arrenca tota la simulació: Gazebo, SLAM, fusió de mapes, navegació, visió i joc.
Simulació	<code>launch/simulation.launch</code> , <code>worlds/</code> , <code>models/</code>	Defineix l'entorn virtual, els robots i el model de la bandera.
SLAM i mapa	<code>shared_slam.launch</code> , <code>map_merge.launch</code> , <code>params/gmapping.yaml</code>	Construeix mapes locals i els fusiona en <code>/merged_map</code> .
Navegació	<code>slam_navigation.launch</code> , <code>params/*slam*.yaml</code>	Configura <code>move_base</code> , <code>costmaps</code> , planificador global i DWA.
Visió	<code>flag_detector_node.cpp</code> , <code>red_flag_detector.cpp</code> , <code>vision.yaml</code>	Detecta la bandera vermella i estima la seva posició en el mapa.
Coordinació	<code>robot_coordinator.py</code> , <code>laser_obstacle_filter.py</code>	Evita bloquejos entre robots i impedeix que l'altre robot embruti el mapa.
Joc CTF	<code>slam_frontier_explorer_ctf.py</code>	Implementa exploració, aproximació a la bandera, captura i persecució.

Una decisió important és separar el sistema en namespaces. El robot 1 publica en topics com `/robot1/scan` o `/robot1/map`, mentre que el robot 2 fa servir `/robot2/scan` i `/robot2/map`. Sense aquesta separació, seria molt fàcil que els topics, frames o action servers es barregessin.

3.1 Inventari complet del repositori

Per evitar que la memòria quedi només com una explicació general, aquesta secció recull tots els elements que apareixen al repositori actual i explica quin paper tenen. La justificació principal de l'estructura és separar responsabilitats: els fitxers de `launch/` descriuen com s'arrenca el sistema, `params/` conté els valors ajustables, `scripts/` agrupa la lògica Python, `src/` i `include/` contenen la part C++, i `worlds/` i `models/` descriuen l'entorn simulat.

Taula 2: Fitxers de configuració general i documentació

Fitxer	Explicació i justificació
<code>package.xml</code>	Defineix el paquet ROS <code>ctf_navigation</code> . Declara dependències de missatges, TF, navegació, Gazebo, GMapping, OpenCV i TurtleBot3. És necessari perquè catkin sàpiga què ha d'estar disponible en compilació i execució.
<code>CMakeLists.txt</code>	Compila la llibreria de visió <code>ctf_vision</code> i l'executable <code>flag_detector_node</code> . També instal·la scripts Python, launch files, paràmetres, mons i models. La decisió de compilar només el detector C++ és coherent perquè la resta de coordinació activa està implementada en Python.
<code>README.md</code>	Dona una entrada ràpida al projecte. No substitueix la memòria, però és útil perquè algú pugui executar la simulació sense llegir tot el document.

Taula 3: Launch files de la simulació

Fitxer	Explicació i justificació
<code>launch/slam_multi_robot_ctf.launch</code>	És el launch principal. Agrupa simulació, SLAM, fusió de mapes, coordinació, navegació, visió i joc. També concentra molts paràmetres CTF, de manera que es pot ajustar el comportament sense tocar el codi.
<code>launch/simulation.launch</code>	Carrega Gazebo, selecciona el món, publica les poses inicials i crea els dos robots i la bandera. Separa la simulació física de la lògica de navegació, cosa que permet canviar el món sense modificar els nodes.
<code>launch/shared_slam.launch</code>	Arrenca els filtres de làser i una instància de GMapping per robot. La justificació és evitar conflictes TF: cada robot construeix el seu mapa en un frame propi.
<code>launch/map_merge.launch</code>	Configura <code>multirobot_map_merge</code> . Utilitza poses inicials conegudes perquè en Gazebo els robots apareixen en posicions controlades. Publica <code>/merged_map</code> , que és el mapa compartit.

Fitxer	Explicació i justificació
<code>launch/slam_navigation.launch</code>	Crea un <code>move_base</code> per robot. Carrega costmaps, planificador global i DWA. Remapeja <code>cmd_vel</code> a <code>cmd_vel_raw</code> perquè el coordinador de robots pugui intervenir abans d'enviar velocitats a Gazebo.
<code>launch/include/spawn_robot_gazebo.launch</code>	Plantilla per crear un TurtleBot3 Waffle Pi dins d'un namespace concret. Evita duplicar el mateix bloc de spawn per a robot1 i robot2.
<code>launch/include/flag_detectors.launch</code>	Inclou dos detectors de bandera, un per robot. Manté simètrica la configuració de visió.
<code>launch/include/flag_detector.launch</code>	Launch d'un detector individual. Carrega <code>vision.yaml</code> i adapta <code>base_frame</code> al namespace del robot.

Taula 4: Scripts Python

Fitxer	Explicació i justificació
<code>scripts/slam_frontier_explorer_ctf.py</code>	Coordinador principal de simulació. Implementa exploració per fronteres, canvi d'estat, aproximació a bandera, captura, persecució i intercepció. Està en Python perquè la lògica de decisió és més flexible i fàcil d'iterar que en C++.
<code>scripts/frontier_exploration_mixin.py</code>	Conté funcions compartides per treballar amb mapes, fronteres, goals i espera de <code>move_base</code> . Separar-ho del coordinador evita que el fitxer principal barregi massa responsabilitats.
<code>scripts/frontier_map_utils.py</code>	Funcions petites de conversió entre mapa i món, càlcul de cobertura i criteri de cel·la lliure. La justificació és centralitzar operacions que es repeteixen constantment.
<code>scripts/laser_obstacle_filter.py</code>	Filtra el LaserScan de cada robot per eliminar l'altre robot abans de GMapping. És necessari perquè el mapa no guardi obstacles dinàmics com si fossin parets.
<code>scripts/robot_coordinator.py</code>	Publica l'altre robot com a <code>PointCloud2</code> i multiplexa <code>cmd_vel_raw</code> cap a <code>cmd_vel</code> . Evita bloquejos i permet donar prioritat en passos estrets.

Fitxer	Explicació i justificació
<code>scripts/robot_cloud_utils.py</code>	Construeix núvols de punts que representen la petjada del robot. Utilitza anells de punts per donar al costmap una àrea ocupada, no només un punt central.
<code>scripts/run_tests.sh</code>	Script auxiliar per llançar proves de simulació amb diferents mons i arguments. No forma part del control en temps real, però documenta una manera ordenada de repetir escenaris de prova.

Taula 5: Codi C++ i headers

Fitxer	Explicació i justificació
<code>src/nodes/flag_detector_node.cpp</code>	Node C++ de visió. Processa imatges, escolta LaserScan, transforma l'estimació al frame <code>map</code> i publica <code>flag_found</code> , <code>flag_estimate</code> i <code>debug_image</code> . S'implementa en C++ perquè el processament d'imatge és sensible al rendiment.
<code>src/vision/red_flag_detector.cpp</code>	Llibreria que fa la segmentació HSV del vermell, neteja morfològica, cerca de contorns i càlcul del centroid. Separar-la del node facilita provar i mantenir la part visual.
<code>include/ctf_navigation/vision/red_flag_detector.hpp</code>	Defineix la configuració del detector i l'estructura de resultat. Dona una interfície clara entre el node ROS i l'algoritme OpenCV.
<code>include/ctf_navigation/vision/laser_utils.hpp</code>	Conté la funció que busca el rang mínim vàlid al voltant d'un bearing. Justifica la fusió càmera-làser: la càmera dona direcció i el làser dona distància.
<code>include/ctf_navigation/common/geometry.hpp</code>	Funcions bàsiques de distància, normalització angular i conversió yaw-quaternion. Són utilitats necessàries en navegació i eviten duplicar càlculs.
<code>include/ctf_navigation/common/tf_helper.hpp</code>	Embolica consultes TF i transformacions de poses amb control d'errors. TF és una font habitual de fallades; per això convé tenir funcions robustes i missatges de warning.
<code>include/ctf_navigation/common/markers.hpp</code>	Helpers per crear marcadors RViz. Són útils per visualitzar bandera, bases o punts rellevants sense repetir codi de Marker .

Fitxer	Explicació i justificació
<code>include/ctf_navigation/ common/move_base_client.hpp</code>	Wrapper C++ per enviar goals a <code>move_base</code> . En el stack actual el coordinador principal és Python, però aquest header documenta i suporta components C++ relacionats.
<code>include/ctf_navigation/ common/slam_wait.hpp</code>	Utilitats per esperar mapa o final d'exploració. Són importants en demos perquè molts errors venen d'arrencar el joc abans que el mapa sigui usable.

Taula 6: Recursos de simulació i visualització

Fitxer	Explicació i justificació
<code>worlds/ctf_world.world</code>	Arena principal de CTF: parets exteriors, paret central amb pas, obstacles i columnes. Serveix per provar exploració, evasió i persecució en un entorn amb decisions de camí.
<code>worlds/nine_rooms_world. world</code>	Món de nou habitacions. Estressa més l'exploració per fronteres perquè força els robots a descobrir habitacions a través de portes.
<code>worlds/hallway_world.world</code>	Passadís amb particions. És útil per provar coordinació entre robots en espais estrets.
<code>worlds/hallway_obstacles_ world.world</code>	Variant del passadís amb obstacles addicionals. Prova el comportament de DWA i dels costmaps quan hi ha desviaments locals.
<code>models/ctf_flag.urdf</code>	Model de la bandera. Ha de ser visible, vermell i físicament coherent amb Gazebo. El color està pensat perquè el detector HSV pugui identificar-lo.
<code>rviz/slam_multi_robot_ctf. rviz</code>	Configuració de RViz per veure mapa fusionat, costmaps, plans i marcadors. És una eina de depuració, no només de presentació.

Taula 7: Fitxers de paràmetres

Fitxer	Explicació i justificació
<code>params/gmapping.yaml</code>	Ajusta el SLAM de cada robot: rang del làser, resolució del mapa, nombre de partícules, llindars de matching i freqüència d'actualització.

Fitxer	Explicació i justificació
params/move_base_slam.yaml	Defineix la freqüència de control, la freqüència de planificació, paciències, retries i comportaments de recuperació de <code>move_base</code> .
params/costmap_common_slam.yaml	Conté el footprint, les fonts d'obstacles i la inflació comuna. És compartit pel costmap local i global per mantenir una mateixa idea de la mida del robot.
params/costmap_global_slam.yaml	Configura el costmap global sobre <code>/merged_map</code> . Serveix per calcular camins llargs coherents amb el mapa fusionat.
params/costmap_local_slam.yaml	Configura una finestra local mòbil de 4.5 m. Aquesta finestra permet reaccionar a obstacles propers sense recalculer tot l'entorn.
params/planner_global_slam_nav.yaml	Ajusta <code>GlobalPlanner</code> : Dijkstra, tolerància de goal, costos i possibilitat de planificar prop d'espai desconegut.
params/planner_local_dwa_slam.yaml	Ajusta <code>DWAPlanerROS</code> : velocitats, acceleracions, toleràncies, mostreig de trajectòries i pesos de cost.
params/vision.yaml	Defineix els llindars HSV, l'àrea mínima del blob vermell, la finestra angular del làser i si es publica imatge de depuració.

4 Execució de la simulació

4.1 Requisits

La simulació està pensada per ROS Noetic i Gazebo, habitualment sobre Ubuntu 20.04. També necessita els paquets de TurtleBot3, la pila de navegació de ROS, GMapping, `multirobot_map_merge`, OpenCV i `cv_bridge`. La variable d'entorn `TURTLEBOT3_MODEL` ha d'estar configurada com a `waffle_pi`, ja que el model utilitzat és el TurtleBot3 Waffle Pi.

Abans d'executar la pràctica cal carregar l'entorn de ROS i el workspace catkin. En una instal·lació típica, la seqüència és carregar `/opt/ros/noetic`, compilar amb `catkin_make` i carregar `devel/setup.bash`. La comanda de llançament utilitza el nom del paquet, és a dir, `ctf_navigation`.

Les dependències principals queden declarades a `package.xml`. Això és important perquè el projecte no només depèn de Python o C++, sinó de molts blocs de ROS: missatges, TF, `actionlib`, navegació, costmaps, OpenCV, Gazebo i el model TurtleBot3. Una instal·lació típica per preparar l'entorn seria:

Instal·lació orientativa de dependències

```
sudo apt update
sudo apt install ros-noetic-desktop-full
sudo apt install ros-noetic-turtlebot3 ros-noetic-turtlebot3-simulations
sudo apt install ros-noetic-navigation ros-noetic-gmapping
sudo apt install ros-noetic-multirobot-map-merge
sudo apt install ros-noetic-global-planner ros-noetic-dwa-local-planner
sudo apt install ros-noetic-cv-bridge ros-noetic-image-transport
```

La instal·lació exacta pot variar segons el laboratori, però aquests paquets reflecteixen les dependències que realment apareixen al repositori. Per exemple, `gmapping` construeix els mapes individuals, `multirobot_map_merge` fusiona aquests mapes, `move_base` executa la navegació, i `cv_bridge` connecta les imatges ROS amb OpenCV.

Compilació del workspace

```
cd ~/catkin_ws
catkin_make
source devel/setup.bash
export TURTLEBOT3_MODEL=waffle_pi
```

4.2 Comanda principal

La simulació completa s'executa amb la comanda `roslaunchctf_navigation slam_multi_robot_ctf.launch`. Aquesta ordre arrenca tots els components necessaris. Per defecte es carrega el món `ctf_world`, però el launch permet seleccionar altres mons, com

```
nine_rooms_world,  
hallway_world o hallway_obstacles_world.
```

Execució bàsica de la simulació

```
source /opt/ros/noetic/setup.bash  
source ~/catkin_ws/devel/setup.bash  
export TURTLEBOT3_MODEL=waffle_pi  
roslaunch ctf_navigation slam_multi_robot_ctf.launch
```

També es poden modificar les posicions inicials dels robots i de la bandera amb arguments de launch. Per exemple, els arguments `flag_x` i `flag_y` permeten moure la bandera sense editar cap fitxer del món. Aquesta flexibilitat és útil per fer proves repetides i comprovar si el sistema continua funcionant quan la bandera no està sempre al mateix lloc.

Exemple amb bandera reposicionada

```
roslaunch ctf_navigation slam_multi_robot_ctf.launch  
flag_x:=1.5  
flag_y:=-2.5
```

5 Arquitectura de la simulació

5.1 Visió global

La simulació es pot entendre com una cadena de transformació d'informació. Gazebo genera un món físic virtual. D'aquest món surten dades de sensors: làser, càmera, odometria i TF. A partir d'aquestes dades, GMapping construeix mapes locals i `map_merge` els fusiona. Paral·lelament, el detector visual intenta trobar la bandera. El coordinador CTF rep mapa, deteccions i posicions dels robots, i decideix quin objectiu ha de tenir cada robot. Finalment, `move_base` converteix aquests objectius en trajectòries i velocitats.

5.2 Diagrama de nodes

La Figura 1 mostra el graf complet de comunicació ROS del sistema multi-robot. Aquest diagrama permet veure amb més detall com es connecten els nodes i topics principals del projecte durant l'execució. A diferència del diagrama general anterior, aquí apareixen també els topics de càmera, odometria, costmaps, plans de navegació, comandes de velocitat i topics interns de `move_base`.

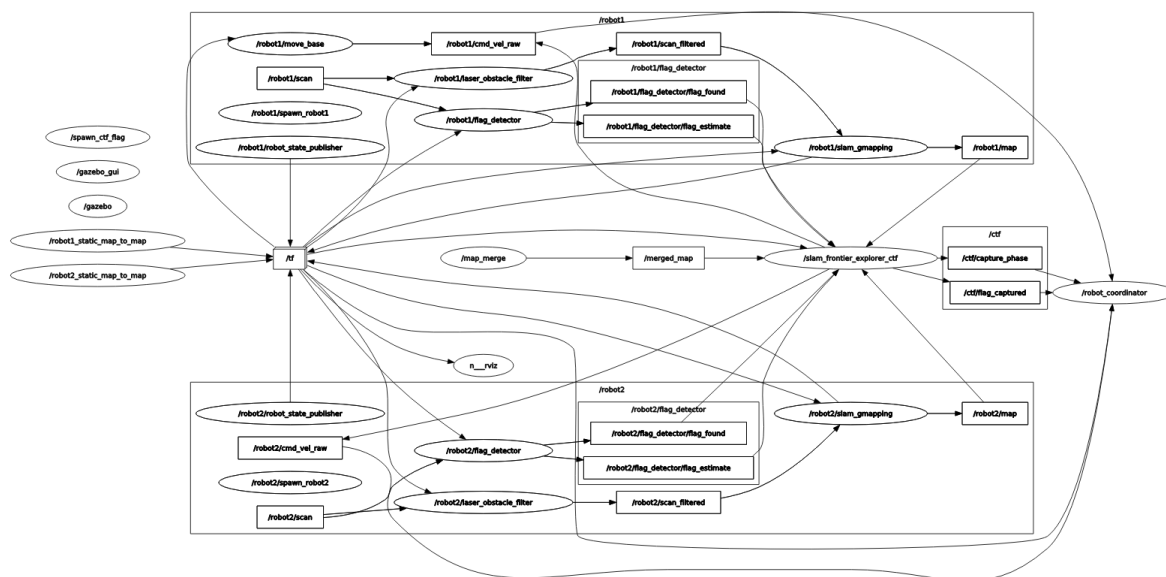


Figura 1: Diagrama de nodes de la simulació renderitzat des de Mermaid

El diagrama està organitzat principalment en dos blocs, un per a `robot1` i un altre per a `robot2`. Cada robot disposa dels seus propis sensors, topics de moviment, detector de bandera, node de SLAM i instància de navegació. Aquesta separació mitjançant namespaces és imprescindible perquè els dos robots puguin funcionar alhora sense que els seus topics es barregin. Per exemple, `/robot1/scan` i `/robot2/scan` representen lectures làser diferents, i `/robot1/move_base` i `/robot2/move_base` corresponen a dos action servers de navegació independents.

A l'esquerra del diagrama apareix Gazebo, que actua com a simulador físic del sistema. Gazebo publica informació dels robots simulats, com les lectures del làser, les imatges de la càmera, els `joint_states` i l'odometria. També rep les comandes finals de velocitat a través dels temes `/robot1/cmd_vel` i `/robot2/cmd_vel`. Això tanca el bucle de simulació: Gazebo genera sensors, els nodes ROS calculen decisions i finalment aquestes decisions es tradueixen en moviment dins del món simulat.

Una part important del graf és la comunicació relacionada amb el làser. Cada robot publica el seu LaserScan original, que passa pel node `laser_obstacle_filter`. Aquest filtre genera els topics `/robot1/scan_filtered` i `/robot2/scan_filtered`. La seva funció és evitar que un robot incorpori l'altre al mapa com si fos un obstacle estàtic. Aquestes lectures filtrades són utilitzades pels nodes de SLAM `/robot1/slam_gmapping` i `/robot2/slam_gmapping`, que publiquen els mapes locals `/robot1/map` i `/robot2/map`.

Els mapes locals es combinen mitjançant el node `/map_merge`, que genera el mapa global compartit `/merged_map`. Aquest mapa fusionat és un element central del sistema, perquè permet que el coordinador CTF prengui decisions a partir de la informació explorada pels dos robots. A més, el mapa també alimenta les instàncies de `move_base`, que necessiten una representació de l'entorn per calcular trajectòries segures.

El graf també mostra la part de visió artificial. Cada robot publica imatges de la seva càmera, tant en format normal com comprimit. Els nodes `/robot1/flag_detector` i `/robot2/flag_detector` utilitzen aquesta informació, juntament amb el làser, per detectar la bandera vermella i estimar-ne la posició. El resultat es publica en topics com `/robot1/flag_detector/flag_found` i `/robot1/flag_detector/flag_estimate`. El primer indica si la bandera ha estat detectada, mentre que el segon proporciona una estimació espacial que pot ser utilitzada pel coordinador del joc.

Les instàncies de `move_base` tenen un paper molt rellevant en el diagrama. Per a cada robot apareixen topics associats al planificador global, al planificador local DWA, als costmaps i als plans generats. El planificador global calcula una ruta sobre el mapa, mentre que `DWAPlanerROS` genera les velocitats locals que permeten seguir aquesta ruta evitant obstacles. Els topics de costmap, footprint i plans són especialment útils per depurar problemes de navegació, perquè permeten saber si un robot no es mou per culpa d'un obstacle, d'un goal mal situat o d'una trajectòria local inviable.

El node `/slam_frontier_explorer_ctf` apareix al centre funcional del sistema. Aquest node rep el mapa fusionat, les estimacions de bandera i l'estat dels robots, i decideix quin comportament ha de seguir cada robot en cada moment. Durant l'exploració selecciona fronteres del mapa; quan es detecta la bandera, envia objectius d'aproximació; i quan es produeix una captura, activa la fase de persecució. Aquesta informació de joc es publica també en topics com `/ctf/flag_captured` i `/ctf/capture_phase`.

Finalment, el node `/robot_coordinator` actua com a capa de coordinació entre els robots. Rep informació del joc i de la navegació, i pot intervenir en les comandes de velocitat per evitar bloquejos, gestionar prioritats de pas o adaptar el comportament quan s'activa la persecució. D'aquesta manera, el sistema no depèn només de dos navegadors independents, sinó que afegeix una capa superior de coordinació multi-robot.

En conjunt, aquest graf mostra que el projecte està format per diferents bucles connectats: el bucle de sensors i simulació, el bucle de SLAM i fusió de mapes, el bucle de detecció visual, el bucle de navegació i el bucle de decisió del joc. El correcte funcionament del sistema depèn de la integració de tots aquests blocs. Per això, aquest diagrama és especialment útil per entendre l'arquitectura global i per localitzar errors quan algun component no publica, no rep informació o no es sincronitza correctament amb la resta.

5.3 Diagrama de flux funcional

La Figura 2 resumeix el funcionament general del sistema des d'un punt de vista funcional. A diferència del graf complet de ROS, aquest esquema no mostra tots els nodes i topics, sinó el camí principal que segueix la informació dins del projecte. Per això és útil com a visió global abans d'entrar en els detalls de cada component.

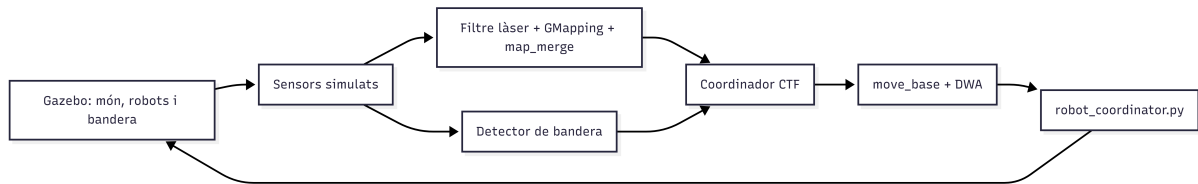


Figura 2: Flux funcional de dades de la simulació

El procés comença a Gazebo, que simula el món, els dos robots i la bandera. Aquest entorn genera les dades que rebrien els robots en una situació real, com les lectures del làser, les imatges de la càmera, l'odometria i les transformacions TF. Per tant, Gazebo actua com la font principal d'informació sensorial i també com el destí final de les ordres de moviment.

A partir dels sensors simulats, el sistema es divideix en dues branques principals. La primera branca correspon al mapatge: les lectures del làser passen pel filtre d'obstacles, després són utilitzades per GMapping per construir mapes locals, i finalment aquests mapes es fusionen mitjançant `map_merge`. El resultat és un mapa global compartit que permet als robots prendre decisions de navegació sobre una representació comuna de l'entorn.

La segona branca correspon a la percepció visual. El detector de bandera utilitza la imatge de la càmera per identificar regions vermelles i combina aquesta informació amb el làser i TF per estimar la posició de la bandera. Aquesta estimació no és només una detecció visual, sinó una posició útil dins del mapa, que després pot ser emprada pel coordinador del joc.

Les dues branques conflueixen en el coordinador CTF. Aquest node rep, d'una banda, el mapa fusionat i, de l'altra, les deteccions o estimacions de bandera. Amb aquesta informació decideix quin comportament ha de seguir cada robot: continuar explorant, apropar-se a la bandera, iniciar la captura o activar la persecució. Així, el coordinador CTF és el bloc que transforma la informació del sistema en decisions de joc.

Un cop presa la decisió, els objectius s'envien a `move_base`. Aquest bloc, juntament amb DWA, s'encarrega de convertir els objectius globals en trajectòries i velocitats locals. El planificador global calcula el camí general fins al goal, mentre que DWA ajusta el moviment instantani per evitar obstacles i seguir una trajectòria viable.

Abans que les velocitats arribin a Gazebo, passen pel node `robot_coordinator.py`. Aquest node afegeix una capa de coordinació entre robots, especialment important quan tots dos comparteixen el mateix espai. Pot modificar o limitar les ordres de moviment per evitar bloquejos, gestionar prioritats de pas o adaptar el comportament durant la fase de persecució.

Finalment, les velocitats resultants tornen a Gazebo i tanquen el bucle. El simulador actualitza la posició dels robots, genera noves lectures de sensors i el procés es repeteix contínuament. Això mostra que el sistema funciona com un bucle tancat de percepció, decisió i acció: els robots observen l'entorn, actualitzen el mapa, detecten la bandera,

decideixen què fer i executen el moviment corresponent.

5.4 Diagrama de la màquina d'estats

La Figura 3 representa la màquina d'estats principal del sistema CTF. Aquest diagrama resumeix com evoluciona el comportament dels robots durant la partida, des de l'exploració inicial fins al final del joc. La màquina d'estats és important perquè el sistema no executa sempre la mateixa lògica: les decisions depenen de si encara s'està explorant, si s'ha detectat la bandera, si ja s'ha capturat o si ha començat la persecució.

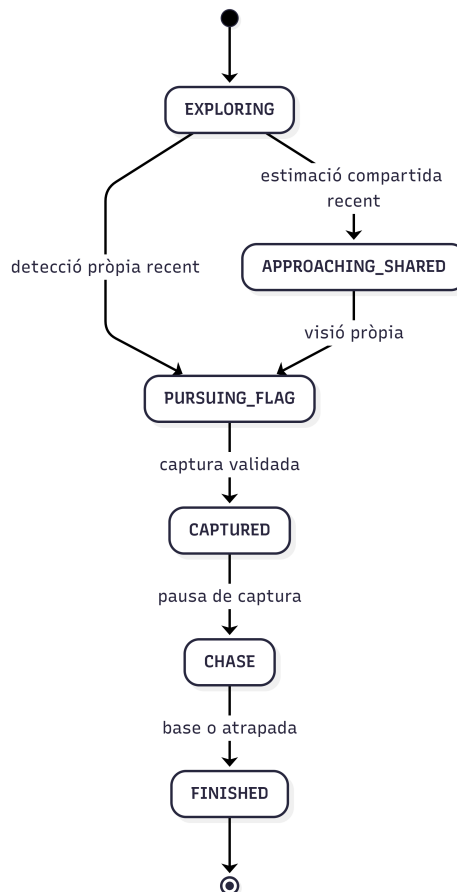


Figura 3: Màquina d'estats del joc CTF

L'estat inicial és **EXPLORING**. En aquesta fase, els dos robots cooperen per explorar l'entorn i construir un mapa compartit. El coordinador busca fronteres en el mapa fusionat i assigna objectius als robots perquè descobreixin zones desconegudes. Mentrestant, els detectors de bandera funcionen en paral·lel, de manera que els robots poden detectar la bandera mentre continuen explorant.

Des de **EXPLORING** hi ha dues possibles transicions relacionades amb la bandera. Si un robot obté una detecció pròpia recent, passa directament a **PURSUING_FLAG**. Això significa que el robot ha vist la bandera amb la seva pròpia càmera i pot començar a apropar-s'hi. En canvi, si només existeix una estimació compartida recent, el sistema pot passar a **APPROACHING_SHARED**. En aquest cas, el robot no ha vist encara la bandera directament,

però utilitza la informació publicada per l'altre robot per moure's cap a la zona on s'ha estimat que es troba.

L'estat `APPROACHING_SHARED` és un estat intermedi. Serveix perquè un robot pugui aprofitar la informació del seu company, però sense considerar encara que té control directe sobre la bandera. La transició cap a `PURSUING_FLAG` només es produeix quan el robot aconseguix visió pròpia. Aquesta condició és important perquè evita que un robot declari una captura basant-se només en una estimació antiga o indirecta.

En l'estat `PURSUING_FLAG`, el robot deixa de prioritzar l'exploració i passa a perseguir la bandera. En aquesta fase, els goals de navegació es generen a partir de l'estimació de posició de la bandera. La captura no es valida immediatament amb una sola detecció, sinó que cal complir unes condicions de proximitat i coherència temporal. Això fa que el sistema sigui més robust davant deteccions puntuals errònies o sorolloses.

Quan la captura es considera vàlida, la màquina passa a l'estat `CAPTURED`. Aquest estat representa el moment en què un robot ha capturat la bandera i es prepara el canvi de fase del joc. És un estat curt, però necessari, perquè permet cancel·lar objectius anteriors, actualitzar els rols dels robots i publicar els topics que indiquen que la bandera ha estat capturada.

Després de la pausa de captura, el sistema entra en l'estat `CHASE`. En aquesta fase el comportament deixa de ser cooperatiu i passa a ser competitiu. El robot que ha capturat la bandera es converteix en portador i intenta tornar a la seva base. L'altre robot es converteix en perseguidor i intenta interceptar-lo o apropar-s'hi prou per atrapar-lo.

Finalment, la partida acaba en l'estat `FINISHED`. Aquesta transició es pot produir per dues raons: o bé el portador arriba a la seva base, o bé el perseguidor aconseguix atrapar-lo abans. Quan s'arriba a aquest estat, el coordinador deixa d'enviar nous objectius de joc i es considera que la partida ha finalitzat.

En conjunt, aquesta màquina d'estats permet estructurar la lògica del projecte de manera clara. Cada fase té unes condicions d'entrada, uns objectius diferents i unes transicions definides. Això evita que el sistema barregi comportaments incompatibles, com explorar i perseguir al mateix temps, i facilita la depuració quan algun canvi de fase no es produeix com s'espera.

6 Nodes de la simulació

6.1 Node de simulació i spawn dels robots

El fitxer `simulation.launch` és l'encarregat de preparar l'entorn virtual. Carrega el món de Gazebo seleccionat, situa la bandera i crea dos TurtleBot3 Waffle Pi. Cada robot es crea dins del seu namespace i té el seu propi `robot_state_publisher`. Això permet que els frames TF quedin separats, per exemple `robot1/base_footprint` i `robot2/base_footprint`.

La bandera es defineix amb el model `ctf_flag.urdf`. En simulació és un objecte vermell visible per la càmera i físicament present al món. El launch guarda també les posicions inicials dels robots i de la bandera al servidor de paràmetres de ROS. Més endavant, el coordinador CTF utilitza aquestes posicions per saber on és la base de cada robot.

6.2 Filtre de làser

El node `laser_obstacle_filter.py` resol un problema específic dels sistemes multi-robot: cada robot pot veure l'altre amb el seu làser. Si aquesta lectura entrés directament a GMapping, el mapa podria acabar contenint obstacles falsos. Aquests obstacles serien el rastre de l'altre robot, no parets reals.

El filtre consulta TF per saber on es troba l'altre robot respecte al frame del làser. A partir d'aquesta posició calcula quins rajos del LaserScan travessen el cercle ocupat pel robot company i substitueix aquelles lectures per infinit. Així GMapping rep una lectura més neta. La sortida és `/robotN/scan_filtered`, que és la lectura que utilitzen tant SLAM com navegació.

Idea principal del filtratge del làser

```
angle_to_obstacle = atan2(other_robot_y, other_robot_x)
half_angle = asin(robot_radius / distance_to_other_robot)

for each laser_beam:
    if beam_angle is inside [angle_to_obstacle - half_angle,
        angle_to_obstacle + half_angle]:
        if measured_range <= distance_to_other_robot + robot_radius:
            measured_range = infinity
```

Aquest fragment resumeix la idea del fitxer `laser_obstacle_filter.py`. No modifica tot el LaserScan, només els rajos que apunten cap a la zona ocupada pel robot company. D'aquesta manera es conserva la informació útil de parets i obstacles reals, però s'elimina l'obstacle dinàmic que no volem guardar al mapa.

6.3 GMapping i mapes individuals

Cada robot executa una instància de `slam_gmapping`. El robot 1 publica `/robot1/map` i el robot 2 publica `/robot2/map`. GMapping utilitza odometria i làser per estimar el moviment del robot i construir un mapa d'ocupació. El mapa és una graella on cada cel·la pot ser lliure, ocupada o desconeguda.

En aquesta simulació es fa servir un frame de mapa per robot. Aquesta decisió és important perquè dues instàncies de GMapping no poden publicar de forma coherent la mateixa transformació `map -> odom`. Separar `robot1/map` i `robot2/map` evita conflictes de TF i deixa que el node de fusió s'encarregui de construir una visió global.

6.4 Fusió de mapes

El node `multirobot_map_merge` rep els mapes locals i publica `/merged_map`. Aquest mapa fusionat és el que utilitzen el coordinador CTF i els costmaps globals. En simulació es treballa amb poses inicials conegudes, perquè Gazebo situa els robots en coordenades controlades. Això simplifica la fusió i redueix la necessitat d'estimar l'alineació dels mapes des de zero.

La fusió és essencial per a la cooperació. Si el robot 1 ha explorat una zona, el robot 2 pot aprofitar aquesta informació a través del mapa fusionat. Això evita que tots dos robots repeteixin exactament la mateixa exploració.

6.5 Navegació amb `move_base`

Cada robot té el seu propi `move_base`. El coordinador CTF no envia velocitats directament, sinó objectius de navegació. `move_base` rep aquests objectius, calcula un camí global sobre el mapa i després genera velocitats locals amb DWA.

El planificador global és `global_planner/GlobalPlanner`. En exploració, el paràmetre `allow_unknown` és especialment important, perquè les fronteres estan just al límit entre zona coneguda i zona desconeguda. Si el planificador no permetés aproximar-se a espai desconegut, el robot tindria problemes per explorar.

El planificador local és `DWAPlannerROS`. DWA genera combinacions de velocitat lineal i angular, simula trajectòries curtes i escull la millor segons el camí global, la proximitat a l'objectiu i la distància als obstacles. En una simulació multi-robot, aquesta decisió local és molt important perquè els robots poden trobar-se en passadissos o portes.

6.6 Coordinador de robots

El node `robot_coordinator.py` actua entre `move_base` i Gazebo. `move_base` publica velocitats crues a `/robotN/cmd_vel_raw`; el coordinador les rep, decideix si s'han de modificar i publica la velocitat final a `/robotN/cmd_vel`.

Durant l'exploració, si els robots estan massa a prop, el robot 2 cedeix el pas al robot 1. Aquesta regla és simple, però funciona bé per evitar bloquejos mutus. A més, el node

publica el robot contrari com un núvol de punts a `/robotN/other_robot_cloud`. Aquest núvol entra al costmap local com un obstacle dinàmic.

Quan la bandera ja ha estat capturada, el comportament canvia. En fase de persecució, el perseguidor ha d'apropar-se al portador; per tant, una evitació massa agressiva impediria el mateix objectiu del joc. Per això el coordinador relaxa part de la lògica d'evitació durant aquesta fase.

6.7 Detector de bandera

El node `flag_detector_node.cpp` s'executa una vegada per robot. Rep la imatge RGB de la càmera i el LaserScan, i publica si veu la bandera i on estima que es troba. La detecció visual es basa en OpenCV i en llindars HSV per al color vermell. Com que el vermell queda dividit als dos extrems de l'escala HSV, el detector utilitza dos intervals de tonalitat.

Segmentació HSV utilitzada pel detector

```
cv::cvtColor(bgr, hsv, cv::COLOR_BGR2HSV);

cv::inRange(hsv, lower_red_1, upper_red_1, mask1);
cv::inRange(hsv, lower_red_2, upper_red_2, mask2);
cv::bitwise_or(mask1, mask2, red_mask);

cv::findContours(red_mask, contours, cv::RETR_EXTERNAL,
cv::CHAIN_APPROX_SIMPLE);
```

Detectar una taca vermella no és suficient. El node calcula el centre horitzontal del blob vermell i el converteix en un angle respecte a la càmera. Després busca un rang del làser en aquella direcció. Amb l'angle i la distància calcula una posició relativa al robot i la transforma al frame global `map` amb TF. El resultat és el topic `/robotN/flag_detector/flag_estimate`.

6.8 Coordinador CTF

El node `slam_frontier_explorer_ctf.py` és el cervell del joc. Manté l'estat global de la partida, escull objectius d'exploració, escolta les deteccions de bandera, decideix si una captura és vàlida i gestiona la fase de persecució.

Internament manté informació com l'estat del joc, el rol de cada robot, les estimacions recents de bandera, les fronteres ja assignades i els objectius que han fallat. Aquesta memòria és necessària perquè el comportament sigui estable. Per exemple, si un objectiu falla, no s'hauria de tornar a intentar immediatament; si un robot ja ha reservat una frontera, l'altre no l'hauria d'agafar al mateix temps.

6.9 Detall intern del coordinador CTF

El coordinador CTF no és només un bucle que envia goals. El fitxer `slam_frontier_explorer_ctf.py` manté diferents capes de memòria perquè el comportament sigui continu entre iteracions. La variable `game_state` descriu la fase global de la partida; `carrier_ns` i `pursuer_ns` indiquen els rols després de la captura; `flag_estimate` i `flag_estimate_time` guarden les deteccions de bandera de cada robot; i `_frontier_claims` i `_blacklist` eviten repetir decisions dolentes.

Aquesta separació és important perquè en ROS els missatges arriben de manera asíncrona. Un detector pot publicar una bandera mentre `move_base` encara està executant un goal de frontera, i el mapa pot actualitzar-se mentre el coordinador està seleccionant un nou objectiu. Per això el codi utilitza locks, timestamps i comprovacions de validesa temporal. No n'hi ha prou amb saber que “alguna vegada” s’ha vist una bandera: cal saber si aquella informació encara és recent i si pertany al robot que està intentant capturar-la.

Decisió simplificada del bucle de cada robot

```
if game_state is CHASE or CAPTURED:
    keep robot idle in the exploration loop
elif robot has a fresh own flag estimate:
    send an approach goal to the flag
elif robot has a fresh shared flag estimate:
    move as support towards the shared estimate
else:
    choose and claim a frontier from the merged map
```

El bucle principal de cada robot funciona en paral·lel mitjançant threads. Aquesta decisió permet que `robot1` i `robot2` triïn i executin objectius de forma independent, però compartint estat global. Si es fes tot en un sol bucle seqüencial, un robot bloquejat esperant `move_base` podria retardar les decisions de l'altre. Amb threads, cada robot pot esperar el seu action server, monitoritzar progrés i recuperar-se d'un bloqueig sense aturar la resta del joc.

Quan un robot rep un goal, el coordinador no espera passivament fins que acabi. El mètode de seguiment comprova distància al goal, estat de l'action server i progrés real del robot. Si la distància no millora durant `progress_timeout`, el goal es considera encallat. Aquesta comprovació és necessària perquè `move_base` pot trigar a declarar un error, sobretot en costmaps que canvien o en zones estretes. El coordinador prefereix detectar abans que un objectiu no funciona i provar una altra frontera.

6.10 Mòduls Python auxiliars

`frontier_exploration_mixin.py` concentra la lògica que no és específica del joc, però que és imprescindible per explorar. Conté la conversió entre mapa i món, la cerca de fronteres, la comprovació de clearance, la construcció de `MoveBaseGoal` i l'espera activa

dels goals. Aquesta divisió fa que el fitxer CTF pugui centrar-se en decisions de joc, mentre que el mixin resol problemes geomètrics i de navegació.

`frontier_map_utils.py` és petit però molt utilitzat. Funcions com `world_to_map`, `map_to_world`, `coverage` o `unknown_count` eviten repetir càlculs i redueixen errors d'índex. En un `OccupancyGrid`, una cel·la no és només una coordenada: depèn de l'origen del mapa, de la resolució i de l'amplada de la graella. Centralitzar aquestes conversions és una decisió de robustesa.

`robot_cloud_utils.py` construeix núvols de punts artificials per representar l'altre robot dins del costmap. Publicar només el centre del robot seria poc útil, perquè el costmap podria deixar passar trajectòries massa properes. En canvi, generar diversos punts al voltant de la petjada crea una ocupació més semblant a un obstacle físic i facilita que DWA el tingui en compte.

`robot_coordinator.py` complementa aquesta idea amb una regla de prioritat. Durant l'exploració, quan els robots són massa propers, un pot cedir el pas. Aquest mecanisme és deliberadament senzill: no intenta resoldre una negociació multi-agent completa, sinó evitar el cas pràctic més habitual, que és que dos robots es quedin empenyent-se o bloquejant-se en una porta. Quan comença la fase de captura o persecució, aquesta regla es relaxa perquè acostar-se a l'altre robot passa a ser part de l'objectiu del joc.

7 Algoritmes i decisions de disseny

7.1 Exploració per fronteres

L'exploració es basa en el concepte de frontera. Una frontera és una cel·la lliure del mapa que toca una cel·la desconeguda. Aquesta idea és intuïtiva: si un robot es mou fins al límit del que coneix, probablement podrà observar una zona nova.

El sistema recorre el mapa fusionat i busca cel·les lliures amb veïnes desconegudes. Després agrupa aquestes cel·les en clústers i calcula un centroide per cada grup. No tots els centroides són bons objectius de navegació, així que abans d'enviar-los a `move_base` es busca una posició propera que tingui clearance suficient. Això evita objectius massa enganxats a parets o obstacles.

Criteri bàsic per detectar una frontera

```
for each cell in occupancy_grid:
    if cell is free:
        if any 4-neighbour cell is unknown:
            mark cell as frontier
```

La frontera escollida no és sempre la més propera. El coordinador combina la distància al robot, la mida del clúster, la distància a l'altre robot, les penalitzacions territorials i, si existeix, la proximitat a una estimació de bandera. Això converteix l'exploració en una heurística multi-objectiu: volem descobrir mapa, però també volem repartir la feina i preparar la captura.

Puntuació conceptual d'una frontera

```
score = distance(robot, frontier) - information_gain(frontier_size)
+ separation_penalty(other_robot) + territory_penalty + flag_bias
```

7.2 Repartiment entre robots

Per evitar que els dos robots vagin al mateix lloc, el sistema utilitza una idea semblant a una partició de Voronoi. Una frontera és més adequada per a un robot si aquell robot hi està més a prop que el seu company. Aquesta regla no és absoluta; hi ha marges i relaxacions perquè el robot no quedi aturat si el mapa ofereix poques alternatives.

També es fa servir una reserva temporal de fronteres. Quan un robot selecciona una frontera, aquesta queda marcada com a seva durant uns segons. Això evita que l'altre robot triï exactament el mateix objectiu en el següent cicle de decisió. Si un objectiu falla, entra temporalment en una blacklist. Aquest mecanisme és molt útil per evitar comportaments repetitius.

7.3 Detecció de color i estimació geomètrica

La detecció de la bandera té dues parts. La part visual respon a la pregunta de si hi ha una regió vermella prou gran a la imatge. La part geomètrica intenta convertir aquella detecció en una posició del mapa.

La segmentació HSV és adequada perquè separa el to del color de la intensitat. En una càmera simulada o real, la il·luminació pot variar, però el to vermell és més estable en HSV que en RGB. Un cop detectat el blob, el detector calcula el seu centre horitzontal. Aquest centre indica si la bandera és a l'esquerra, al centre o a la dreta de la càmera.

El làser aporta la distància. El node busca una lectura vàlida del LaserScan al voltant de l'angle on s'ha vist la bandera. Amb angle i distància es calcula una posició local, i amb TF es transforma al frame `map`. Aquesta estimació no és perfecta, però és suficient per orientar la navegació i activar la fase de captura quan el robot s'hi apropa.

7.4 Condició de captura

La captura no s'activa amb una sola detecció. El robot ha d'estar en fase d'aproximació, ha de tenir una estimació pròpia i recent, ha de continuar veient la bandera, i ha d'estar dins de la distància de captura. A més, la situació ha de persistir un petit interval temporal.

Aquesta decisió redueix falsos positius. En visió artificial és habitual que una detecció puntual sigui sorollosa. Si el sistema declarés captura immediatament, una taca vermella accidental podria canviar tota la partida. La persistència temporal fa que la captura sigui més robusta.

7.5 Persecució i intercepció

Després de la captura, el robot captador es converteix en portador. El seu objectiu és tornar a la base on va començar. L'altre robot passa a ser perseguidor. Aquesta fase és diferent de l'exploració perquè els robots ja no intenten separar-se; ara un vol fugir i l'altre vol apropar-se.

El perseguidor pot anar directament cap al portador o intentar interceptar-lo. La intercepció es calcula sobre la línia entre el portador i la seva base. Si el perseguidor pot arribar a un punt d'aquesta línia abans que el portador, aquell punt pot ser millor que perseguir-lo per darrere. Si els robots ja són molt a prop, el sistema abandona la intercepció i envia el perseguidor directament cap al portador.

La persecució acaba en dues situacions. Si el portador arriba a casa, guanya el portador. Si el perseguidor queda dins de `catch_distance`, es considera que l'ha atrapat i guanya el perseguidor.

8 Paràmetres de simulació

Els paràmetres permeten ajustar el comportament sense recompilar el projecte. En la simulació s'utilitzen principalment fitxers YAML dins de `params/` i arguments del launch principal.

8.1 Paràmetres del launch principal

Taula 8: Arguments més rellevants de `slam_multi_robot_ctf.launch`

Paràmetre	Valor habitual	Interpretació
<code>world</code>	<code>ctf_world</code>	Món de Gazebo que es carrega.
<code>rviz</code>	<code>true</code>	Indica si RViz s'obre automàticament.
<code>world_frame</code>	<code>map</code>	Frame global compartit.
<code>merged_map_topic</code>	<code>/merged_map</code>	Mapa fusionat utilitzat per navegació i exploració.
<code>flag_x</code> , <code>flag_y</code>	depèn del món	Posició inicial de la bandera.
<code>robot1_x</code> , <code>robot1_y</code>	depèn del món	Posició inicial del robot 1.
<code>robot2_x</code> , <code>robot2_y</code>	depèn del món	Posició inicial del robot 2.

8.2 Paràmetres de GMapping

Taula 9: Justificació dels paràmetres principals de GMapping

Paràmetre	Valor	Justificació
<code>maxUrange</code> , <code>maxRange</code>	3.5 m	Limiten el rang útil del làser. En mons petits, usar rangs massa llargs pot introduir lectures llunyanes poc útils i augmentar soroll.
<code>delta</code>	0.05 m	Resolució del mapa. Cinc centímetres és un compromís habitual: prou detall per portes i parets, però sense fer el mapa massa pesat.
<code>particles</code>	30	Nombre de partícules del filtre. Amb dos robots, augmentar molt aquest valor penalitzaria la CPU

Paràmetre	Valor	Justificació
linearUpdate, angularUpdate	0.2 / 0.2	Actualitza el mapa quan el robot s'ha mogut prou. Valors més petits actualitzarien massa sovint; valors més grans farien el mapa menys reactiu.
temporalUpdate	1.0 s	Garanteix actualitzacions encara que el moviment sigui lent. Això ajuda quan el robot gira o queda parcialment bloquejat.
minimumScore	80	Llindar de qualitat del matching làser. Ajuda a evitar actualitzacions dolentes quan les lectures no encaixen bé.
xmin, xmax, ymin, ymax	-6 a 6	Mida inicial

8.3 Paràmetres de move_base

`move_base_slam.yaml` defineix la freqüència de control i la política de recuperació. El controlador local funciona a 12 Hz, suficient per una simulació amb velocitats moderades. El planificador global funciona a 1.5 Hz perquè no cal recalculer el camí complet a cada instant; el cost computacional baixaria poc i, en canvi, podria generar plans massa canviants.

Taula 10: Justificació de paràmetres de `move_base`

Paràmetre	Valor	Justificació
controller_frequency	12.0	Manté una resposta local fluida sense saturar la simulació.
planner_frequency	1.5	Replanifica prou sovint per adaptar-se, però no tant com per provocar oscil·lacions.
planner_patience, controller_patience	8.0	Dona temps al robot per resoldre passos estrets abans de declarar fallada.
max_planning_retries	4	Evita insistir indefinidament en un goal impossible.
oscillation_timeout	5.0	Detecta bloquejos locals relativament ràpid.
clearing_rotation_allowed	false	Evita rotacions de neteja agressives que en multi-robot poden empitjorar bloquejos.

8.4 Costmaps

El costmap comú defineix el footprint del TurtleBot3 Waffle Pi i les fonts d'obstacles. El footprint no és circular perquè el Waffle Pi no ocupa exactament un cercle; representar-lo com un polígon ajuda el planificador a entendre millor l'espai real necessari.

Taula 11: Justificació de costmaps

Paràmetre	Valor	Justificació
footprint	polígon Waffle Pi	Representa millor la base que un radi únic i evita plans massa optimistes.
obstacle_range	2.5 m	Marca obstacles propers, que són els que afecten la navegació local.
raytrace_range	3.5 m	Permet netejar espai lliure fins al rang útil del làser.
observation_sources	scan + other_robot	Combina obstacles del làser i obstacle dinàmic del company.
global_inflation	0.30	Fa que els plans globals prefereixin zones centrades i no freguin parets.
local_inflation	0.15	Manté el robot prou flexible per passar per portes i passadissos estrets.
local_width/height	4.5 m	Finestra local prou gran per anticipar obstacles sense processar tot el mapa.

8.5 Planificador global

`planner_global_slam_nav.yaml` utilitza Dijkstra mitjançant `GlobalPlanner`. El paràmetre més important és `allow_unknown=true`. En exploració, els objectius es troben al límit de l'espai conegut; si el planificador prohibís completament l'espai desconegut, molts goals de frontera serien impossibles encara que fossin correctes.

`default_tolerance=0.5` permet acceptar una posició propera al goal si el punt exacte no és navegable. Això encaixa amb l'exploració per fronteres, on el centroides calculat pot estar prop d'una paret o d'una cel·la desconeguda. El coordinador ja intenta ajustar el goal a una cel·la lliure, però aquesta tolerància dona una segona capa de robustesa.

8.6 DWA local planner

`planner_local_dwa_slam.yaml` limita la velocitat lineal màxima a 0.24 m/s. Aquesta velocitat és conservadora, però adequada per un joc amb dos robots i passadissos estrets. El robot pot retrocedir lleugerament (`min_vel_x=-0.12`), cosa útil quan queda encaixat davant d'un obstacle o del robot company.

Taula 12: Justificació de DWA

Paràmetre	Valor	Justificació
max_vel_x	0.24	Velocitat moderada per reduir xocs i donar temps al costmap.
min_vel_x	-0.12	Permet petites maniobres de retrocés en bloquejos.
max_vel_theta	2.0	Gir ràpid suficient per explorar i orientar la càmera.
xy_goal_tolerance	0.22	Evita exigir precisió impossible en goals propers a fronteres.
sim_time	1.5	Horitzó curt, adequat per entorns petits i obstacles propers.
vx_samples, vtheta_samples	24 / 48	Mostreig ric per trobar trajectòries en espais estrets.
path_distance_bias	32.0	Dona importància a seguir el camí global.
goal_distance_bias	20.0	Manté tendència cap al goal sense ignorar el camí.
occdist_scale	0.06	Penalitza obstacles sense fer el robot excessivament conservador.

Aquest és el tipus de fragment que resumeix la filosofia del fitxer YAML: velocitat moderada, horitzó curt i moltes mostres angulars per poder maniobrar en passadissos estrets.

8.7 Visió

`vision.yaml` està ajustat per una bandera vermella en Gazebo. Els dos rangs HSV són necessaris perquè el vermell apareix als dos extrems del canal hue.

Taula 13: Justificació dels paràmetres de visió

Paràmetre	Valor	Justificació
horizontal_fov	1.085595	Valor del model Waffle Pi; necessari per convertir píxels a bearing.
min_blob_area	150	Evita microdeteccions de soroll o reflexos.
max_detection_distance	5.0	Limita estimacions massa llunyanes, menys fiables amb làser i SLAM.
range_window	0.25	Finestra angular prou ampla per compensar errors de càmera i làser.

Paràmetre	Valor	Justificació
h_low1/h_high1	0–15	Primer interval del vermell en HSV.
h_low2/h_high2	165–180	Segon interval del vermell en HSV.
s_min, v_min	100 / 60	Rebutgen colors poc saturats o massa foscos.
publish_debug	true	Facilita verificar visualment què detecta el node.

8.8 Paràmetres del joc CTF

Els paràmetres del joc apareixen sobretot al launch principal. No són valors arbitraris: intenten equilibrar captura, persecució i navegació. Una distància de captura massa petita faria que el robot intentés xocar amb la bandera; una distància massa gran declararia captures abans d'arribar realment.

Taula 14: Justificació de paràmetres CTF principals

Paràmetre	Valor	Justificació
goal_timeout	45 s	Temps suficient per arribar a fronteres sense quedar bloquejat indefinidament.
min_coverage	0.72	Referència de cobertura per saber si l'exploració és prou avançada.
gain_scale	2.0	Dona preferència a fronteres grans, que aporten més informació.
voronoi_margin	0.15	Evita canvis bruscos de frontera entre robots.
territory_penalty	0.5	Redueix creuaments innecessaris sense imposar una divisió rígida.
frontier_claim_sec	35 s	Reserva temporal d'objectius per evitar duplicació.
flag_capture_distance	0.60 m	Captura física sense exigir contacte amb el cilindre.
catch_distance	0.65 m	Distància raonable per considerar que el perseguidor ha atrapat el portador.
flag_memory_timeout	120 s	Manté estimacions útils durant exploració, però no per sempre.
intercept_enabled	true	Permet persecució predictiva, no només seguir la posició actual.

9 Funcionament complet de la partida

9.1 Inici

Quan s'arrenca la simulació, Gazebo crea el món, els robots i la bandera. Durant els primers segons els nodes de TF, sensors, GMapping i navegació s'estabilitzen. El coordinador CTF espera abans d'enviar objectius, perquè si es comença massa aviat és possible que `move_base` encara no estigui preparat.

9.2 Exploració

Un cop el sistema està actiu, els robots comencen a explorar. El mapa fusionat encara és parcial, així que el coordinador busca fronteres. Cada robot rep un objectiu diferent, normalment en una zona on pot aportar informació nova. Si un robot falla repetidament en arribar a un objectiu, aquell punt queda temporalment bloquejat.

Durant aquesta fase, els detectors de bandera funcionen en paral·lel. Això vol dir que els robots no fan una fase separada de "mirar la bandera"; exploren i miren al mateix temps. Si la bandera entra al camp de visió d'un robot, el coordinador canvia el comportament.

9.3 Aproximació i captura

Quan un robot veu la bandera amb la seva pròpia càmera, deixa de perseguir fronteres i passa a aproximar-s'hi. L'altre robot pot utilitzar l'estimació compartida per apropar-se a la zona, però la captura només és vàlida amb visió pròpia recent. Aquesta regla evita que un robot capturi una bandera que en realitat ja no està veient.

La captura s'activa quan la distància és prou petita i la detecció persisteix. En aquell moment s'aturen els objectius d'exploració i es publiquen els topics de canvi de fase. El robot que ha capturat es converteix en portador; l'altre passa a ser perseguidor.

9.4 Persecució

Durant la persecució, el portador rep objectius cap a la seva base. El perseguidor rep objectius que depenen de la situació. Si està molt a prop, va directament al portador. Si està més lluny, intenta calcular un punt d'intercepció. Els objectius es refresquen perquè la posició dels robots canvia constantment.

La partida acaba quan el portador arriba a casa o quan el perseguidor entra dins de la distància de captura. En tots dos casos, el coordinador atura els objectius i publica que l'exploració/joc ha finalitzat.

10 Execució en robot real

Aquesta secció documenta la versió del projecte executada sobre dos TurtleBot3 Waffle Pi reals. El codi corresponent es troba a la branca `robot_real`, mentre que la part de simulació s'havia treballat a la branca `simulacio\_net`. Tot i que l'arquitectura general es manté, en el robot real ha calgut adaptar diversos components perquè el comportament físic no és tan ideal com a Gazebo.

La diferència més important és la detecció de la bandera. En simulació es podia detectar una bandera vermella amb segmentació de color, però en l'entorn real aquesta estratègia generava falsos positius. Algunes zones amb molta llum, reflexos o saturació de la imatge eren interpretades com a vermell. Per això, en la versió real s'ha substituït la detecció per color per una detecció amb marcador ArUco/AprilTag. Aquesta decisió fa que la bandera sigui més robusta: ja no depèn només del color, sinó d'un patró visual codificat i identificable.

10.1 Diferències principals respecte a la simulació

Taula 15: Canvis principals entre simulació i robot real

Aspecte	Simulació	Robot real
Càmera	Topic de Gazebo <code>camera/rgb/image_raw</code>	Topic real de <code>cv_camera</code> : <code>cv_camera/image_raw/compressed</code> .
Detecció de bandera	Segmentació HSV del color vermell	Marcador ArUco/AprilTag, id 3, diccionari <code>DICT_APRILTAG_36h11</code> .
TF	Gazebo publica una cadena coherent de frames	Cal reconstruir explícitament <code>robotN/odom -> robotN/base_footprint</code> i <code>base_footprint -> base_link -> base_scan</code> .
Fusió de mapes	<code>multirobot_map_merge</code> amb poses inicials conegudes	Fusió pròpia per TF amb <code>tf_map_merge.py</code> , més estable en multi-PC real.

Aspecte	Simulació	Robot real
Navegació	Sensors ideals i latència baixa	Costmaps i <code>move_base</code> s'endarrereixen uns segons per esperar SLAM, TF i scans.
Coordinació	Posicions obtingudes principalment amb TF global	Coordinació relativa basada en spawn inicial + odometria de rodes per evitar problemes quan els mapes SLAM deriven.

10.2 Launch principal del robot real

El launch principal és `launch/real\_multi\_robot\_ctf.launch`. Aquest fitxer substitueix el launch de Gazebo i arrenca el sistema complet sobre robots físics.

Les poses inicials per defecte situen els robots separats tres metres:

Poses inicials en robot real

```
<arg name="robot1_x" default="-1.50"/>
<arg name="robot1_y" default="0.0"/>
<arg name="robot1_yaw" default="0.0"/>

<arg name="robot2_x" default="1.50"/>
<arg name="robot2_y" default="0.0"/>
<arg name="robot2_yaw" default="0.0"/>
```

Aquests valors no només serveixen com a referència visual. També són utilitzats per la fusió de mapes, pel coordinador de robots i per definir la base de cada robot durant el joc. En robot real és important que aquestes coordenades coincideixin amb la col·locació física inicial.

10.3 Detecció de bandera amb ArUco/AprilTag

La detecció de color vermell funcionava en simulació perquè la il·luminació de Gazebo és controlada. En el robot real, però, la càmera pot veure reflexos, zones cremades per la llum, objectes vermellorsos o canvis de balanç de blancs. Això feia que el detector HSV pogués confondre una zona il·luminada amb la bandera.

Per solucionar-ho, el detector s'ha reimplementat amb ArUco/AprilTag. El fitxer `src/vision/aruco\_flag\_detector.cpp` substitueix l'antic detector de color i busca un marcador amb un identificador concret. La configuració real es troba a `params/vision\_real.yaml`:

Configuració de visió en robot real

```
camera_topic: ""
compressed_camera_topic: "cv_camera/image_raw/compressed"
scan_topic: "scan"

max_image_age: 0
drop_delayed_frames: false
flag_hold_sec: 2.0

aruco_marker_id: 3
aruco_dictionary: 20      # DICT_APRILTAG_36h11

publish_debug: true
```

El paràmetre `aruco\_marker\_id=3` indica quin marcador representa la bandera. El paràmetre `aruco\_dictionary=20` correspon al diccionari `DICT\_APRILTAG\_36h11`. Aquest tipus de marcador és més robust que una taca de color perquè codifica informació dins del patró. Per tant, el detector no només pregunta si hi ha una zona vermella, sinó si hi ha un marcador vàlid amb l'identificador esperat.

El detector continua publicant els mateixos topics que en simulació:

Topics publicats pel detector

```
/robotN/flag_detector/flag_found
/robotN/flag_detector/flag_estimate
/robotN/flag_detector/debug_image
```

Això és important perquè el coordinador CTF no necessita canviar completament la seva lògica: continua rebent una estimació de bandera, però aquesta estimació ara prové d'un detector més fiable.

10.4 Estimació de posició de la bandera

El detector ArUco dona una posició en píxels dins de la imatge, però el joc necessita una posició en el mapa. Per això el node `flag\_detector\_node` combina càmera, làser i TF. Primer calcula el centre del marcador a la imatge; després converteix aquest centre en un angle horitzontal utilitzant el camp de visió de la càmera; finalment busca una lectura vàlida del làser al voltant d'aquell angle.

10.5 Topics de càmera en el robot real

En Gazebo, la càmera es publicava habitualment a `camera/rgb/image\_raw`. En el TurtleBot3 real, el bringup de la càmera fa servir `cv\_camera`, i el topic esperat és:

Topic de càmera en robot real

```
/robot1/cv_camera/image_raw/compressed  
/robot2/cv_camera/image_raw/compressed
```

Per això existeix el launch `launch/real\_robot\_flag\_vision.launch`, que permet provar només la visió sense executar tot el joc. Aquest launch és útil abans de fer una prova completa, perquè permet comprovar que les càmeres publiquen, que el marcador es detecta i que la imatge de debug és correcta.

Prova independent de la visió

```
roslaunch ctf_navigation real_robot_flag_vision.launch  
rostopic echo /robot1/flag_detector/flag_found  
rostopic echo /robot2/flag_detector/flag_found
```

10.6 Adaptacions de TF

En simulació, Gazebo i el model del robot proporcionen una cadena TF bastant estable. En robot real, sobretot amb dos robots i diversos ordinadors, és més fàcil que apareguin frames sense namespace o arbres TF separats. Per això s'ha afegit `launch/real_robot_tf.launch`.

Aquest launch publica o reconstrueix la cadena:

Cadena TF esperada en robot real

```
map  
-> robot1/map  
    -> robot1/odom  
        -> robot1/base_footprint  
            -> robot1/base_link  
                -> robot1/base_scan  
  
map  
-> robot2/map  
    -> robot2/odom  
        -> robot2/base_footprint  
            -> robot2/base_link  
                -> robot2/base_scan
```

El node `scripts/odom\_tf\_broadcaster.py` escolta `/robotN/odom` i republiques la transformació amb noms explícits: `robotN/odom -> robotN/base\_footprint`. Això evita un problema habitual: que el bringup del robot publiqui frames relatius com `odom` o `base\_footprint` sense namespace, cosa que trenca la integració multi-robot.

10.7 Navegació i coordinació entre robots

El node `robot\_coordinator.py` també s'ha adaptat. En robot real utilitza el paràmetre `use_odom_relative=true`. Amb aquesta opció calcula la posició relativa entre robots a partir de la pose inicial i l'odometria de rodes. Això és més estable per publicar l'altre robot com a obstacle dinàmic dins del costmap local.

Coordinació relativa en robot real

```
<param name="use_odom_relative" value="true"/>
<rosparam param="robot1_spawn">[-1.5, 0.0, 0.0]</rosparam>
<rosparam param="robot2_spawn">[ 1.5, 0.0, 0.0]</rosparam>
```

Durant l'exploració, si els robots s'apropen massa, el robot 2 cedeix el pas al robot 1. Aquesta regla evita bloquejos físics en passadissos o zones estretes. Quan comença la fase de captura o persecució, aquesta evitació es relaxa perquè l'objectiu del joc pot ser precisament acostar-se a l'altre robot.

10.8 Ordre d'execució recomanat

L'ordre d'execució és més crític en robot real que en simulació. Abans de llançar el joc cal assegurar-se que tots els ordinadors comparteixen el mateix `ROS\_MASTER\_URI`, que els robots publiquen odometria, làser i càmera, i que els rellotges estan sincronitzats.

Comprovacions prèvies

```
# En tots els ordinadors
sudo timedatectl set-ntp true
date

# Amb els bringups ja actius
rostopic list | grep robot1
rostopic list | grep robot2
rostopic hz /robot1/odom
rostopic hz /robot1/scan
rostopic hz /robot1/cv_camera/image_raw/compressed
```

Un cop els robots publiquen correctament, es pot llançar el sistema complet:

Execució completa en robot real

```
source ~/catkin_ws/devel/setup.bash
export TURTLEBOT3_MODEL=waffle_pi

roslaunch ctf_navigation real_multi_robot_ctf.launch
```

11 Conclusió

En conclusió, aquest projecte ha permès desenvolupar i integrar un sistema multi-robot capaç de jugar a atrapa la bandera combinant SLAM, fusió de mapes, navegació autònoma, detecció visual i coordinació entre robots. La part de simulació ha estat fonamental per validar l'arquitectura general en un entorn controlat i per ajustar els principals paràmetres del sistema abans de provar-lo amb robots reals.

L'execució en robot real ha mostrat que el pas de Gazebo a l'entorn físic requereix adaptacions importants. Problemes com la sincronització de TF, la qualitat dels sensors, la il·luminació, la xarxa o la deriva de l'odometria fan que el comportament real sigui menys ideal. Per això s'han introduït canvis com la detecció de la bandera amb ArUco i una gestió més explícita dels frames i de la coordinació entre robots.

Un dels punts més rellevants del projecte és que el funcionament correcte no depèn només d'un node concret, sinó de la comunicació entre tots els blocs. Si el mapa no es fusiona bé, si TF no és coherent o si `move_base` no rep informació estable, la lògica del joc també es veu afectada. Això evidencia la importància de dissenyar una arquitectura modular, però ben connectada.

Tot i les limitacions observades, el projecte mostra una base funcional i extensible per a comportaments multi-robot més complexos.